

Security Vulnerability Analysis of JWT Storage Methods in Browser Environments and Proposal of Authentication Architectures by Storage Method: Based on OWASP ZAP Analysis Results

브라우저 환경에서의 JWT 저장 방식 보안 취약점 분석 및 상황별 인증 아키텍처 제안: OWASP ZAP 분석 결과를 중심으로

Joun Heejae¹, Cho Sungbin², Cha Mookwan³, Lee Taek⁴

전희재¹, 조성빈², 차무관³, 이택^{4*}

¹ Student, Division of CS and Engineering, Sun Moon University, Korea, jounhj02@gmail.com

² Student, Division of CS and Engineering, Sun Moon University, Korea, choys4444@gmail.com

³ Student, Division CS and Engineering, Sun Moon University, Korea, tjddlck1@gmail.com

⁴ Professor, Division of CS and Engineering, Sun Moon University, Korea, comtaek@gmail.com

초록:

현대 웹 애플리케이션에서 JWT(JSON Web Token)는 상태를 유지하지 않는(Stateless) 인증의 표준으로 자리 잡았으나, 발급된 토큰을 브라우저의 어느 공간(Local Storage, Session Storage, Cookies)에 저장할 것인지에 대한 보안적 논의는 여전히 실무적 난제로 남아있다. 본 연구는 브라우저 환경에서 JWT를 저장하는 주요 방식들의 보안 특성과 구현 난이도를 비교 분석하고, 서비스 환경에 적합한 인증 아키텍처를 제안하는 것을 목적으로 한다. 이를 위해 JWT 인증을 사용하는 GitHub 오픈소스 프로젝트를 수집하고, 자동화 취약점 진단 도구인 OWASP ZAP을 활용하여 저장 방식별 취약점(CSP 미설정, CORS 오류, 쿠키 보안 속성 누락 등)을 분석하였다. 분석 결과, Local Storage 방식은 구현 편의성이 높으나 XSS(Cross-Site Scripting) 공격을 통한 직접적인 토큰 탈취 위험에 취약한 것으로 나타났다. 반면, Cookies 방식은 HttpOnly 속성 등을 통해 XSS 방어에는 유리하지만 CSRF(Cross-Site Request Forgery) 방어 및 교차 출처(Cross-Origin) 설정의 복잡도가 증가하는 한계를 보였다. 이러한 분석 결과를 바탕으로 본 연구는 서비스의 보안 민감도, 구현 난이도, 출처(Origin) 구성 환경(동일 출처 및 교차 출처)을 종합적으로 고려하여 소규모 서비스, 일반 웹 서비스, 교차 출처(Cross-Origin) 환경, 보안 민감 서비스 4가지 상황 맞춤형 JWT 인증 아키텍처를 제안한다. 본 연구는 단순한 저장 방식의 우위를 넘어 실무 개발자가 보안성과 구현 현실성을 균형 있게 검토하여 최적의 인증 아키텍처를 설계하는 데 기여할 수 있다.

핵심어: JWT(JSON Web Token), 인증 아키텍처(Authentication Architecture), 브라우저 저장소(Browser Storage), 취약점 분석(Vulnerability Analysis), OWASP ZAP 웹 보안(Web Security)

목차

초록.....	1
목차.....	2
1장. 서론.....	3
2장. 선행연구.....	4
2.1 JWT(JSON Web Token).....	4
2.2 JWT 저장 방식 분류.....	5
2.3 SOP(Same Origin Policy).....	7
2.4 Access Token과 Refresh Token.....	8
3장. 연구 방법.....	10
3.1 분석할 리포지토리 수집.....	10
3.2 OWASP ZAP 중심 취약점 분석.....	10
3.3 ZAP 중심 취약점 분석	12
3.4 저장 방식별 보안 구현 난이도 평가 기준.....	12
4장. 분석 및 분석결과.....	15
4.1 Local Storage 기반 JWT 프로젝트의 ZAP 취약점 분석 결과.....	15
4.2 Cookies 기반 JWT 프로젝트의 ZAP 취약점 분석 결과.....	16
4.3 저장 방식별 비교 분석.....	17
4.4 상황별 인증 아키텍처 제안.....	21
5장. 결론.....	25
참고문헌.....	26

1. 서론

현대 웹 애플리케이션 아키텍처는 마이크로서비스 아키텍처(MSA, Microservice Architecture)의 확산과 RESTful API¹의 보편화에 따라, 서버의 상태를 유지하지 않는(Stateless) 인증 방식인 JWT(JSON Web Token)를 사실상의 표준으로 채택하고 있다. JWT는 서버의 확장성을 극대화하고 다양한 클라이언트 환경을 지원할 수 있다는 강력한 이점을 제공하지만, 상태를 클라이언트가 관리해야 한다는 구조적 특성상 토큰의 탈취 및 위·변조 위협에 지속적으로 노출된다.

성공적인 인증 이후 발급된 JWT를 클라이언트(웹 브라우저)의 어느 공간에 저장할 것인가에 대한 논쟁은 여전히 실무적 난제로 남아있다. 현재 널리 사용되는 Local Storage 및 Session Storage 기반의 저장 방식은 구조적으로 XSS(Cross-Site Scripting) 공격에 노출될 경우 저장된 토큰이 직접 탈취될 수 있는 위험을 가진다. 반면, HttpOnly 속성을 적용한 쿠키 기반 저장 방식은 XSS(Cross-Site Scripting)로 인한 토큰 탈취 위험을 크게 감소시킬 수 있으나, 쿠키가 요청 시 자동으로 전송된다는 특성 때문에 CSRF 방어를 위한 SameSite 속성, CSRF Token 등의 추가적인 보안 대책이 요구된다.

소프트웨어 개발에 있어 속도보다 중요한 것은 올바른 방향성이다. 그럼에도 불구하고 다수의 현업 프로젝트에서는 프론트엔드 프레임워크와의 연동 편의성이나 교차 출처(Cross-Origin) 문제 해결의 용이성만을 이유로 보안적 트레이드오프를 면밀히 검토하지 않은 채 Local Storage 기반의 아키텍처를 관행적으로 채택하는 경우도 다수 존재한다. 이는 궁극적으로 시스템의 신뢰성을 훼손하고 사용자에게 보안 위협을 전가하는 결과를 초래한다. 따라서 단순히 특정 저장 방식의 우위를 논하는 것을 넘어, 실제 오픈소스 프로젝트 분석을 바탕으로 각 저장 방식의 보안 특성을 비교할 필요가 있다.

본 연구는 브라우저 환경에서 JWT를 저장하는 주요 방식들의 보안 특성을 비교 분석하는 것을 목적으로 한다. 이를 위해 JWT 인증 방식을 사용하는 GitHub 오픈소스 프로젝트를 수집하고, OWASP ZAP을 활용하여 CSP Header 미설정, Cross-Domain Misconfiguration, 쿠키 보안 속성 미흡 등 JWT 인증 구조와 연관될 수 있는 취약점을 분석하였다. 또한 탐지된 취약점을 저장 방식별 보안 특성과 구현 난이도 관점에서 해석하고, Local Storage, Cookie, Hybrid 방식의 장단점을 비교하였다. 나아가 서비스 환경과 보안 요구 수준에 따라 적용 가능한 상황별 JWT 인증 아키텍처를 제안하고자 한다.

¹ HTTP 프로토콜과 URI를 기반으로 자원(Resource)을 식별하고, 데이터를 주고받기 위한 설계 아키텍처(REST)를 충실히 따른 애플리케이션 인터페이스

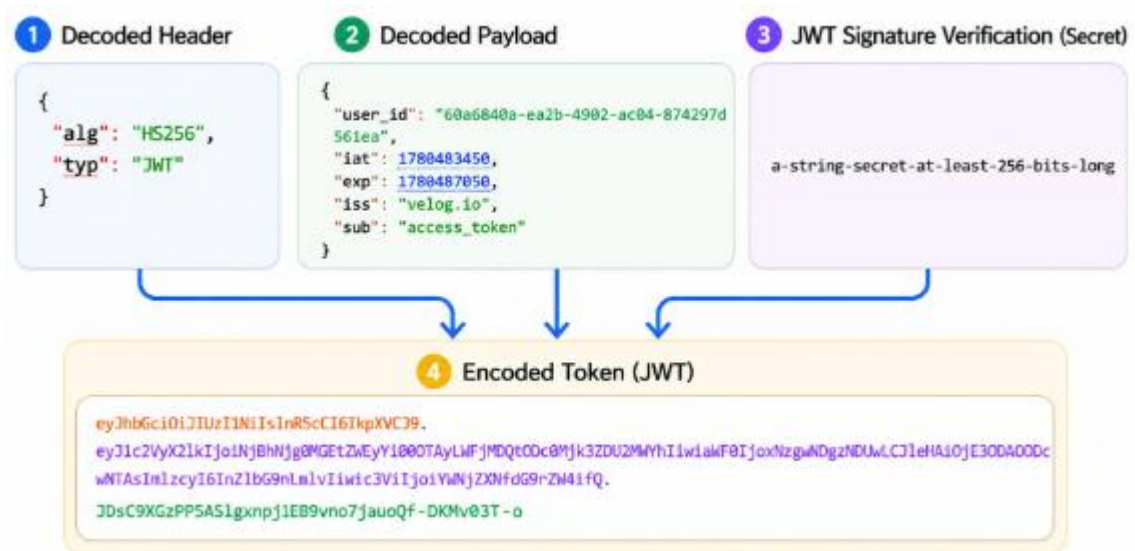
2. 선행연구

2.1 JWT(JSON Web Token)

JWT(JSON Web Token)는 웹 애플리케이션에서 사용자 인증과 권한 부여를 효과적으로 관리하기 위해 사용되는 보안 기술 중 하나로, JSON 객체를 사용해 두 개체 간에 정보를 안전하고 간결하게 전송하기 위한 웹 표준이다. 인증에 필요한 정보를 JSON 형태로 저장한 후 전자서명을 통해 무결성을 보장한 토큰을 이용해 사용자 인증을 수행한다.

JWT는 서버의 부담을 줄이기 위해 고안된 토큰 기반 인증 방식으로, 서버가 사용자 상태(Session)를 별도로 저장하지 않는 무상태(Stateless) 인증을 지원한다. 따라서 서버는 세션 저장소를 유지할 필요가 없어 확장성과 분산 환경에 유리하다. JWT는 RFC 7519 표준에 정의되어 있다.

JWT의 구조는 header, payload, signature의 세 부분으로 구성되어 있다.[1] 각 부분을 마침표(.)로 구분하여 하나의 문자열을 이룬다. [그림 1]은 JWT의 구조를 나타낸다. Header와 Payload는 Base64 Url 방식으로 인코딩되며, Signature는 Header와 Payload 그리고 서버의 비밀키를 이용하여 생성된다. 최종적으로 세 부분은 마침표(.)로 연결되어 하나의 JWT를 구성한다.



[Figure 1] JWT Structure

[그림 1] JWT 구조

Header(헤더)는 일반적으로 토큰 유형(typ)과 서명에 사용되는 알고리즘(alg)정보를 포함한다.[2] 대표적인 알고리즘으로는 HS256, RS256, ES256 등이 있다. Header 정보는 Base64 Url로 인코딩 되어 있다.

Payload(페이로드)는 Claim(클레임)정보를 포함한다. 클레임은 Registered Claim, Public

Claim, Private Claim 세 종류가 있다. Registered Claim 중에서는 iss(발급자), sub(사용자 식별자), exp(만료시간)같은 사용자 정보가 저장되어 있어 서버 또는 클라이언트가 필요한 정보를 쉽게 얻을 수 있다. Payload 정보는 Base64 Url로 인코딩될 뿐 암호화하지 않으므로 누구나 디코딩하여 내용을 확인할 수 있다. 따라서 password와 같은 민감한 정보는 저장하지 않아야 한다.

Signature(서명)는 Header와 Payload를 결합한 값에 서버의 비밀키(Secret Key)를 이용하여 생성된 전자서명 값이다. 서버는 이를 이용해 토큰의 위변조 여부를 검증할 수 있으며, Signature 값이 일치하지 않을 경우 해당 토큰은 변조된 것으로 판단한다.

JWT는 서버 측 상태 저장 없이 인증을 수행할 수 있다는 장점을 가지지만, 토큰이 클라이언트 측에 저장되는 특성으로 인해 저장 방식에 따라 다양한 보안 위협에 노출될 수 있다.[3] 특히 LocalStorage, SessionStorage, Cookies의 저장 위치에 따라 XSS(Cross-Site Scripting), CSRF(Cross-Site Request Forgery), Token Theft(토큰 탈취)와 같은 취약점의 위험도가 달라질 수 있다. 따라서 JWT의 저장 방식에 대한 보안 분석은 안전한 인증 아키텍처 설계를 위해 중요한 연구 과제이다.

2.2 JWT 저장 방식 분류

브라우저 환경에서 JWT 저장 방식은 3가지로 구분한다. Cookies, Local Storage, Session Storage 방식이다.



Figure 2. JWT Storage Locations in a Browser as Viewed in Developer Tools

[그림 2] 개발자 도구에서 확인한 브라우저의 JWT 저장소

2.2.1 Cookies

웹 환경에서 통신은 세션이 없는 무상태성을 기반으로 한다. 쿠키는 이를 극복하기 위한 기술로 루 몬톨리(Lou Montulli)에 의해 1994년에 개발되었다.[4] 쿠키를 통한 세션 지원 절차는 다음과 같다.

1. 서버는 Set-Cookie을 Http Response header에 포함하여 클라이언트에게 전송한다.
2. 클라이언트는 브라우저의 Storage 영역인 Cookies에 수신한 쿠키를 저장한다.
3. 매 Http Request마다 쿠키의 특성에 따라 자동적으로 쿠키 정보가 header에 포함되어 전송된다.
4. 서버는 매 Http Request header의 쿠키 정보를 읽고 세션을 지원한다.

쿠키는 key-value 형식이며 Set-Cookie: <cookie-name>=<cookie-value>로 정의한다. 설정할 옵션을 세미콜론(;)을 구분자로 하여 덧붙여 전송한다.

[표 1] Optional Settings in Cookie-Set

[표 1] 쿠키 설정 가능한 옵션

옵션 코드	비고
SameSite=<samesite-value>	CSRF 공격을 방어하며, 옵션으로 Strict(동일한 사이트 요청에만 Cookie 전송), Lax(이미지,프레임 및 사이트간 요청 차단), None(사이트간 요청, 같은 사이트 요청 모두 전송)
HttpOnly	XSS 방어를 위한 기능으로 JavaScript가 Document.cookie로 접근하는 것을 금지
Secure	HTTPS 연결에서만 전송
Domain=<domain-value>	쿠키를 읽을 수 있는 도메인 설정이며, 생략시 현재 문서의 URL 호스트를 기본 값으로 설정
Partitioned	서드파티 쿠키의 사이트간 추적 방지
Path=<path-value>	전송될 URL 경로 범위 지정
Max-Age<number>	생명주기를 초단위로 설정하며 Expires보다 우선순위를 가짐
Expires=<date>	생명주기를 설정하며, 생략시 클라이언트가 종료될 때 삭제

[표 2] Criteria for Assessing the Security Importance of Cookie Options

[표 2] 쿠키 옵션 별 보안 중요도 평가 기준

중요도	옵션 항목
High ²	SameSite, HttpOnly
Medium ³	Secure, Domain, Partitioned, Path
Low ⁴	Max-Age, Expires

² XSS 및 CSRF 공격 방어에 필수적이다.

³ 다른 취약점과 결합되어 공격 시도를 할 수 있다.

2.2.2 Local Storage

Local Storage는 명시적으로 삭제하기 전까지 영구적으로 데이터가 보존되는 비휘발성 저장소이다. JavaScript를 통해 데이터 접근이 용이하여, HTTP 요청 헤더 (Authorization: Bearer <token>)에 토큰을 삽입하는 등의 구현 편의성이 높다. 하지만 악성 스크립트의 접근을 원천 차단하는 보안 기제가 부재하다. 이로 인해 교차 사이트 스크립팅(XSS) 공격에 노출될 경우 저장된 데이터가 탈취될 수 있는 치명적인 취약점을 지닌다.

2.2.3 Session Storage

Session Storage는 특정 브라우저 탭(Tab) 내에서만 데이터가 유효하며, 해당 탭이 종료될 때 데이터가 함께 소멸되는 휘발성 저장소이며 생명주기가 짧은 특징을 가지고 있다. 한 탭에서만 유효하다는 특징 때문에, 탭이 바뀌면 로그아웃되는 사용자 불편의 문제가 존재한다. Local Storage와 마찬가지로 JavaScript를 통한 접근이 가능하므로 XSS 공격에 취약하다.

2.3 SOP(Same-Origin Policy)

웹 콘텐츠에서 출처(Origin)는 URL의 스키마⁵, 도메인, 포트로 정의된다.[5] 두 객체 간의 스키마, 도메인, 포트가 같다고 할 때 동일 출처(Same-Origin)라고 하며, 이 중 하나라도 다른 경우 교차 출처(Cross-Origin)라고 한다. “https://example.com”에 대한 출처 일치 비교 예시는 다음과 같다.

[Table 3] Examples of Origin Matching for "https://example.com"

[표 3] “https://example.com”에 대한 출처 일치 비교 예시

출처	출처 일치 여부	비고
http://example.com:443	X	스키마(http)가 다름
https://example.com/app1/index.html	O	스키마,호스트, 포트 같음
https://example.com:8080	X	https의 기본 포트(443)와 다름
https://myapp.example.com	X	호스트가 다름

SOP(Same-Origin Policy)는 어떤 출처에서 불러온 문서나 스크립트가 다른 출처에서 가져온 리소스와 상호작용⁶할 수 있는 방법을 제한하는 브라우저의 중요한 보안 메커니즘이

⁴ 공격 방어에 미치는 영향이 상대적 작다.

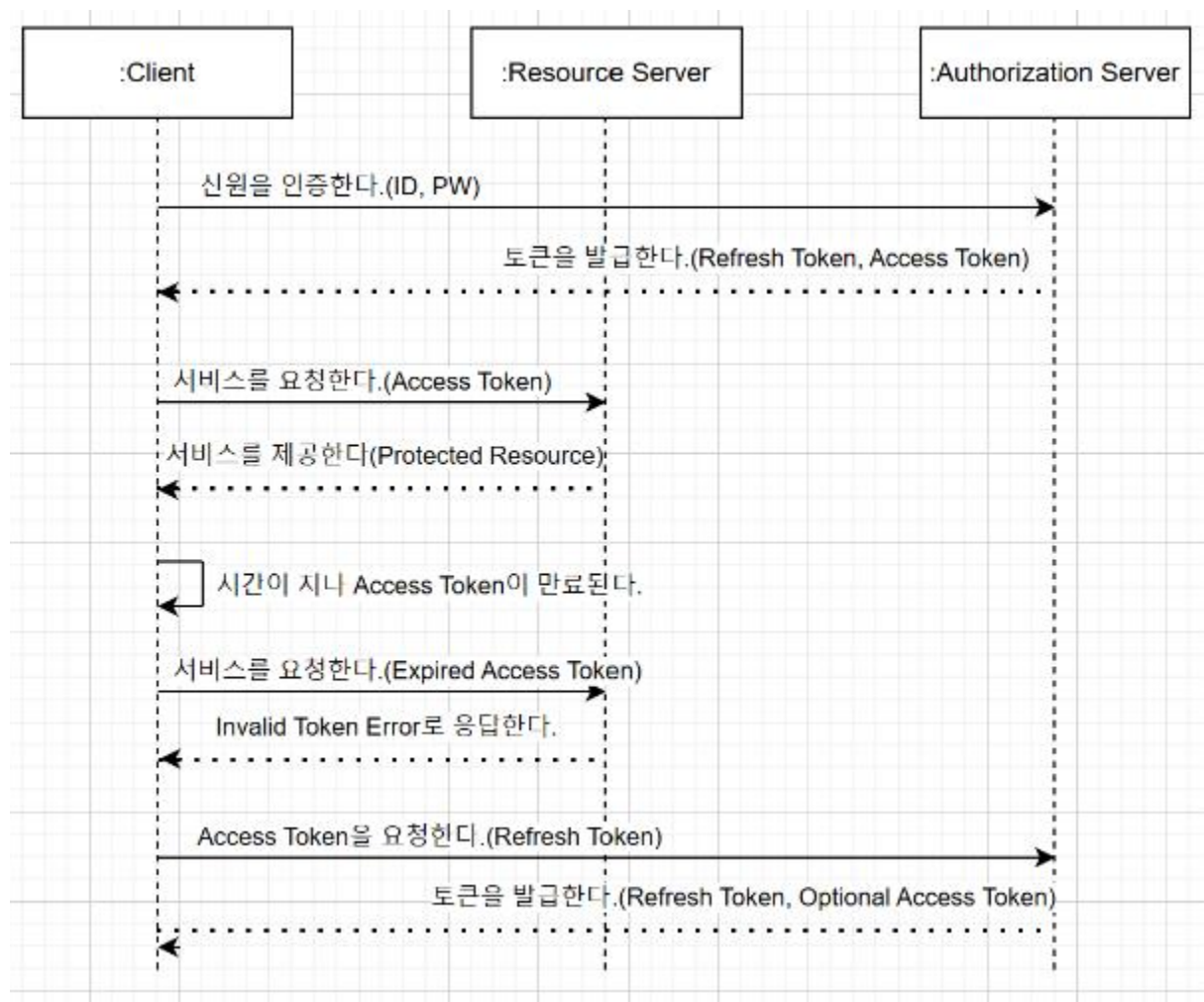
⁵ http, https, ftp 등

⁶ Html의 기본적인 링크 이동(<a>), 폼 제출(<form>), 이미지 로드(), 스크립트 로드

다.[6] SOP는 Cookies, Local Storage, Session Storage 등에 저장된 인증 정보가 다른 출처의 스크립트에 의해 무단으로 접근되지 않도록 보호한다.

2.4 Access Token과 Refresh Token

JWT와 같은 토큰 기반 인증은 서버에 상태를 저장하지 않는(Stateless) 장점이 있으나, 토큰 자체가 탈취당하면 만료 전까지 서버가 이를 통제할 수 없다는 치명적인 보안 결함을 갖는다. 이를 방어하기 위해 2012년에 발표한 OAuth 2.0 RFC 6749의 Access Token과 Refresh Token[7]으로 보안설계를 진행할 수 있다.



[Figure 3] Sequence Diagram for Refreshing an Expired Access Token

[그림 3] 만료된 Access Token 갱신 시퀀스 다이어그램

Access Token은 사용자 인증 완료 후 발급하고, 사용자가 API에 접근할 수 있는 권한을 부여받는다. 일반적으로 5~30분 정도의 짧은 수명을 가지며 공격자가 Access Token을 탈취할 경우 계정을 도용할 수 있다. Refresh Token은 Access Token 만료 시 재발급 용도이

(`<script></script>`)는 가능하다. 다만 `fetch()` 등 통한 다른사이트의 응답 읽기가 제한된다.

며 Access Token에 비해 상대적으로 긴 수명을 가지고 있다.

Access Token과 Refresh Token은 저장 위치에 따라 서로 다른 보안 위험에 노출된다. 일반적으로 Access Token은 메모리 또는 Web Storage에 저장되며, Refresh Token은 HttpOnly 설정이 된 채로 쿠키에 저장된다. 따라서 JWT 저장 방식에 대한 보안성 분석은 단순한 토큰 저장 위치의 문제가 아니라 Access Token과 Refresh Token의 역할 분리 전략 과도 밀접한 관련이 있다. [8]

[표 4] Access Token과 Refresh Token의 역할 및 특성 비교

[Table 4] Comparison of the Roles and Characteristics of Access Token and Refresh Token

Token 종류	역할	보편적인 저장 위치	만료 기간
Access Token	API 접근 권한	메모리, Web Storage	짧은 수명
Refresh Token	Access Token 재발급	HttpOnly Cookie	긴 수명

3. 연구방법

본문 내용. 본 연구에서는 JWT 인증 방식을 사용하는 GitHub 오픈소스를 Local Storage, Session Storage, Cookies 총 3 집단으로 구분하고, OWASP ZAP을 통한 Cross-Site Request Forgery, XSS, JWT 취약점 분석을 진행한다.

해당 통계를 바탕으로 각 저장방식 별 취약점이 실제 탈취 및 권한 도용으로 이어질 수 있는 위험도를 수치화하여 상황 맞춤형 JWT 인증 아키텍처를 제안한다.

3.1 분석할 리포지토리 수집

보다 빠른 분석 대상 수집을 위해 자동화 검색 프로그램⁷을 만들어 수집하였다. 자동화 검색 프로그램은 다음과 같은 기준으로 작동한다.

1. JWT 인증을 사용하는가?⁸
2. 스타 수가 50 보다 큰가?
3. Docker-compose 를 지원하는가?

리스트업 된 리포지토리를 최종적으로 Local Storage, Session Storage, Cookies 저장소로 직접 분류를 하였다. 최종 선정된 저장소 별 수는 다음과 같다.

[표 5] JWT 저장소 별 리포지토리 선정수

[Table 5] Number of Selected Repositories by JWT Storage Method

Token 저장 위치	건
LocalStorage	3
SessionStorage	0
Cookie	3

3.2 OWASP ZAP 중심 취약점 분석

해당 연구에서는 OWASP ZAP을 통해 취약점 분석을 진행한다. 취약점 분석시 사용한 Scan Policy는 [표 6] 와 같다. 취약점 분석시 Threshold값은 Medium으로 설정하였다. Threshold는 취약점 탐지 규칙의 민감도를 결정하며 오탐과 탐지율의 균형을 고려하여 Medium으로 결정하였다.

[표 6] 분석시 사용한 Scan Policy

[Table 6] Using Scan Policy

Client Browser	크로스 사이트 스크립팅(DOM기반)
Injection	Cross Site Scripting(Persistent)
	Cross Site Scripting(Reflected)

⁷ https://github.com/ChaMooKwan/jwt-storage-security-analysis/blob/main/Automative_Searching_for_Github.ipynb

⁸ 레포지토리 검색 키워드로 'JWT', 레포지토리 내 코드 검색 키워드로 'jwt.sign'를 사용하였다.

Miscellaneous	JWT Scan Rule
---------------	---------------

3.2.1 크로스 사이트 스크립팅(DOM기반)

Client Browser에서 실행되는 취약점으로, URL의 Fragment(#) 또는 Query String 값이 JavaScript에 의해 DOM에 삽입될 때 발생한다. 서버 응답과 무관하게 클라이언트 측 코드에서 XSS payload가 실행되는지 확인한다.

3.2.2 Cross Site Scripting

Stored XSS의 경우 게시물, 댓글, 프로필 등 서버에 저장되는 입력값에 악성 스크립트를 삽입한 후 다른 사용자가 해당 데이터를 조회할 때 실행되는지 검증한다. Reflected XSS의 경우 요청 파라미터에 삽입된 스크립트가 서버 응답에 즉시 반영되어 실행되는지를 기반으로 취약 여부를 확인한다.

3.2.3 JWT Scan Rule

JWT Scan Rule은 ZAP에 포함된 JWT 분석 애드온으로 다음과 같은 취약점 테스트를 진행한다.

3.2.3.1 None Algorithm attack

JWT의 alg 헤더를 “none”으로 변경하고 서명(Signature)을 제거한 토큰을 서버에 전송하여, 서명 검증이 정상적으로 수행되지 않는 경우 인증 우회 가능성을 확인한다.

3.2.3.2 Algorithm Confusion attack

JWT에서 RS256과 HS256과 같은 비대칭키/대칭키 알고리즘 간 검증 로직이 잘못 구현된 경우 발생하는 취약점이다. 공격자가 alg 헤더를 HS256으로 변경하고 서버의 공개키를 대칭키처럼 사용하도록 유도할 경우 서명 검증이 우회되는지 확인한다.

3.2.3.3 Trusting JWT provided with the Token

Cisco의 node-jose 0.11.0 이전 버전에서 발생한 취약점으로 공격자가 JWT토큰을 자신의 공개키로 재서명하고 JWT 헤더의 자신의 공개키를 같이 보냈을때 서버에서 공격자의 키로 서명을 검증하는 동작을 하는지 검증한다.

3.2.3.4 Empty JWT

JWT 자체 취약점보다는 개발자의 예외처리 실수로 인해 발생하는 문제로 불완전한 JWT 토큰이나 빈 JWT토큰을 보내 서버에서 응답하는지 확인한다.

3.2.3.5 Null byte Injection attack

주로 C계열 언어에서 나타나는 문제로 W0을 만났을 때 문자열이 끝난것으로 취급하는 문제 이를 JWT토큰에 활용, role과 같은 속성에 adminWn0000user 과 같이 삽입하여 동작이 달라지는 지를 검증한다.

3.3 ZAP을 통한 취약점 분석

ZAP을 통한 취약점 분석은 JWT Scan Rule을 사용하기 위해 [그림 4]과 같이 JWT 토큰이 Local Storage 또는 Cookies에 저장된 상태에서 진행하였다.

Name	Value	D...	Path	Expi...	Size	Http...	Sec...	Sam..
_spree_jwt	eyJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkIjoxMTY4...	d...	/	202...	254	✓	✓	Lax
_spree_refresh_...	3w3RTVTeNthGDUz9oSZbvoL1	d...	/	202...	44	✓	✓	Lax

[Figure 4] Stored JWT

[그림 4] 저장된 JWT

Alert type	Risk	Count
Content Security Policy (CSP) Header Not Set	Medium	2 (15.4%)
Cross-Domain Misconfiguration	Medium	150 (1,153.8%)
Missing Anti-clickjacking Header	Medium	2 (15.4%)
Application Error Disclosure	Low	1 (7.7%)

[Figure 5] Analysis Results of JWT_Authentication_React Vulnerabilities

[그림 5] JWT_Authentication_React 취약점 분석 결과

ZAP에서 자동화 공격을 통해 [그림 5]과 같이 보안 취약점을 위험도 별로 수집하고 분석을 수행한다.

3.4 저장 방식별 보안 구현 난이도 평가 기준

본 연구에서는 JWT 저장 방식의 보안 취약점 탐지 분석에서 그치지 않고, 실제 개발 과정에서 안전한 인증 구조를 구현하기 위해 요구되는 보안 설정의 복잡도와 구현 부담을 함께 고려하였다. 연구에 들어가기 전 보안 구현 난이도라는 단어에 대해 정의하도록 하겠다. 보안 구현 난이도란 특정 JWT 저장 방식을 웹 애플리케이션에 적용할 때 보안성을 확보하기 위해 개발자가 추가적으로 설정하거나 구현해야 하는 항목의 복잡도를 의미한다.

해당 평가는 개발 시간이나 코드 라인 수를 직접 측정한 것이 아니라, 공식 문서와 보안 권고에서 요구하는 보안 설정 항목, 설정 간 의존성, 그리고 인증 흐름의 복잡도를 기준으로 Low, Medium, High의 3단계로 구분하였다. Local Storage와 Cookies를 같이 사용하는 경우는 Hybrid 방식이라고 하겠다.

[Table 7] Criteria for Evaluating Security Implementation Complexity by JWT Storage Method

[표 7] JWT 저장 방식별 보안 구현 난이도 평가 기준

평가 항목	Local Storage	Same-Origin Cookie	Cross-Origin Cookie	Hybrid
토큰 저장 구현	JavaScript 저장 및 조회	Set-Cookie 설정 필요	Set-Cookie 및 Cross-Origin 설정 필요	Access Token과 Refresh Token 분리 필요
요청 포함 방식	Authorization Header에 수동 포함	Cookie 자동 전송	Credentials 기반 Cookie 전송	Access Token 요청 포함 및 Refresh 요청 분리
XSS 방어 부담	High ⁹	Low ¹⁰	Low ¹¹	Medium ¹²
CSRF 방어 부담	Low ¹³	Medium ¹⁴	High ¹⁵	Medium~High ¹⁶
CORS 설정 복잡도	Medium	Low~Medium	High	High
토큰 갱신 로직	Low	Medium	Medium	High
종합 구현 난이도	Low	Medium	High	High

평가 기준은 다음과 같다. 첫째, 토큰 저장 및 요청 포함 방식이다. Local Storage 방식은 JavaScript를 통해 토큰을 저장하고 Authorization Header에 직접 포함할 수 있어 구현이 비교적 단순하다. 반면 Cookies 저장 방식은 서버의 Set-Cookie 응답 헤더와 브라우저의 Cookies 저장 및 자동 전송 메커니즘을 함께 고려해야 한다.

둘째, XSS 방어 설정 필요성이다. Local Storage와 Session Storage는 JavaScript를 통한 접근이 가능하므로, XSS 공격이 발생할 경우 저장된 토큰이 직접 노출될 수 있다. 따라서 CSP 적용 입력값 검증, 출력 인코딩 등 클라이언트 측 스크립트 실행을 통제하기 위한 방어 설정의 중요도가 높다. 반면 Cookies 방식은 HttpOnly 설정으로 JavaScript 접근을 제한할 수 있으므로 토큰 직접 탈취 위험을 줄일 수 있으나, HttpOnly 속성이 누락될 경우 이러한 장점이 약화된다.

셋째, CSRF 방어 설정 필요성이다. 방식은 브라우저가 요청 시 Cookie를 자동으로 전송하는 구조이므로, 사용자가 의도하지 않은 요청이 인증된 상태로 전송될 수 있다. 이를 완화하기 위해 SameSite 속성, CSRF Token, Origin 등의 추가 방어 기법을 고려해야 한다. 반

⁹ JavaScript로 접근이 가능하므로, 악성 스크립트가 서버에 삽입되었을 때, 공격 벡터 지점이 매우 많아짐에 따라 높다고 평가하였다.

¹⁰ Cookie의 경우 HttpOnly 옵션 설정만으로 JavaScript의 접근을 차단할 수 있어 낮다고 평가하였다.

¹¹ 위 각주와 동일하다.

¹² JavaScript로 Local Storage에 저장되어 있는 Access Token에 접근 가능하나 짧은 만료시간 설정으로 인해 Local Storage보다 높은 중간으로 평가하였다.

¹³ Local Storage 방식은 토큰을 Authorization Header에 수동으로 포함하는 구조이므로, 브라우저가 인증 정보를 자동 전송하는 Cookie 방식에 비해 CSRF 위험이 상대적으로 낮다고 평가하였다.

¹⁴ Cookie의 Optional 설정인 SameSite=Strict, Lax 옵션 설정만으로 CSRF 공격을 완화할 수 있어 중간으로 평가하였다.

¹⁵ Cross-Origin Cookie 방식에서는 SameSite, Secure, credentials, CORS 허용 Origin 설정이 함께 요구되므로 방어 부담이 높다고 평가하였다.

¹⁶ Same-Origin Hybrid나 Cross-Origin Hybrid에 따라 중간과 높음으로 평가한다.

면 Local Storage 방식은 토큰을 Authorization Header에 수동으로 포함하는 구조이므로 Cookie 기반 인증에 비해 CSRF 위험은 상대적으로 낮다.

넷째, CORS 및 SOP 설정 복잡도이다. 프론트엔드와 백엔드가 서로 다른 Origin으로 구성된 환경에서는 브라우저의 Same-Origin Policy(SOP)를 우회하기 위해 CORS 설정이 필요하다. 특히 Cookie 기반 인증에서는 인증 정보를 포함한 Cross-Origin 요청을 처리하기 위해 클라이언트 측의 credentials 설정과 서버 측의 Access-Control-Allow-Credentials, Access-Control-Allow-Origin 헤더 설정이 함께 요구된다. 또한 Cookie의 SameSite, Secure 등의 속성도 적절히 구성되어야 한다. 이러한 설정이 올바르게 적용되지 않을 경우 브라우저가 Cookie를 정상적으로 처리하지 못하거나, 저장된 Cookie가 Cross-Origin 요청에 포함되지 않아 인증 기능이 정상적으로 동작하지 않을 수 있다.

다섯째, 토큰 갱신 및 로그아웃 처리의 복잡도이다. Access Token과 Refresh Token을 분리하여 사용하는 Hybrid 방식은 토큰 탈취 시 피해 범위를 줄일 수 있다는 장점이 있으나, Access Token 만료 처리, Refresh Token 재발급 API, 로그아웃 시 토큰 무효화, Refresh Token Rotation 등의 추가 로직이 필요하다. 따라서 Hybrid 방식은 보안성은 높지만 구현 난이도 또한 높은 방식으로 평가하였다.

이러한 기준을 바탕으로 본 연구에서는 Local Storage 방식은 구현 난이도를 Low, Same-Origin Cookie 방식은 Medium, Cross-Origin Cookie 방식과 Hybrid 방식은 High로 평가하였다. 이 기준은 특정 저장 방식의 절대적 우열을 판단하려는 의도가 아니고, 저장 방식 선택 시 보안성과 구현 현실성을 함께 고려하기 위한 기준으로 활용된다.

4. 분석 및 분석결과

본 연구에서는 JWT 인증 방식을 사용하는 GitHub 오픈소스 프로젝트를 대상으로 OWASP ZAP 기반 취약점 분석을 수행하였다. 분석 대상은 JWT 저장 방식에 따라 Local Storage 기반 프로젝트와 Cookie 기반 프로젝트로 구분하였다. Session Storage 기반 프로젝트의 경우 실제 서비스 환경에서 사용 사례를 확보하지 못하여 제외하였다.

분석 결과, 저장 방식과 관계없이 CSP(Content Security Policy) 미설정, 크로스 도메인 구성 오류, 클릭재킹 방지 헤더 누락 등의 취약점이 반복적으로 발견되었다. 이는 웹 애플리케이션 전반에서 공통적으로 발생하는 보안 설정 미흡 문제로 판단된다.

그러나 동일한 취약점이 존재하더라도 JWT 저장 위치에 따라 공격자가 토큰에 접근할 수 있는 방식에는 차이가 존재하였다. Local Storage 방식은 JavaScript를 통한 직접 접근이 가능하여 XSS 공격 발생 시 토큰 노출 위험이 존재하는 반면, HttpOnly 설정이 되어 있는 Cookie 방식은 JavaScript 접근이 제한되어 토큰 직접 탈취 가능성이 상대적으로 낮은 것으로 분석되었다.

이에 따라 본 장에서는 저장 방식별로 발견된 취약점을 분석하고, 각 취약점이 JWT 인증 구조에 미치는 영향을 비교한다.

4.1 Local Storage 기반 JWT 인증 프로젝트의 ZAP 취약점 분석 결과

4.1.1 PERN-Store¹⁷ 프로젝트

- 탐지 취약점 : CSP Header Not Set, Cross-Domain Misconfiguration
- JWT 관련 취약점 수 : 2개

PERN-Store 프로젝트는 Local Storage에 Access Token을, Cookie에 Refresh Token을 저장하여 JWT 인증 기능을 제공한다. OWASP ZAP 분석 결과 CSP Header 미설정, Cross-Domain Misconfiguration 오류 취약점이 확인되었다. JWT 관련 취약점은 총 2종으로 분류하였다.

4.1.2 MERN E-Commerce¹⁸ 프로젝트

- 탐지 취약점 : CSP Header Not Set, Cross-Domain Misconfiguration
- JWT 관련 취약점 수 : 2개

MERN E-Commerce 프로젝트는 JWT를 Local Storage에 저장하여 인증을 유지하는 구조로 사용된다. OWASP ZAP 분석 결과 CSP Header 미설정과 Cross-Domain Misconfiguration 취약점이 확인되었다. JWT 관련 취약점은 총 2종으로 분류하였다.

¹⁷ <https://github.com/dhatguy/pern-store>

¹⁸ <https://github.com/mohamedsamara/mern-ecommerce>

4.1.3 MySQL Registration Passport¹⁹ 프로젝트

- 탐지 취약점 : CSP Header Not Set, Cross-Domain Misconfiguration
- JWT 관련 취약점 수 : 2개

MySQL Registration Passport 프로젝트는 Local Storage를 활용한 JWT 인증 기능을 지원한다. OWASP ZAP 분석 결과 CSP Header 미설정과 Cross-Domain Misconfiguration 취약점이 확인되었다. JWT 관련 취약점은 총 2종으로 분류하였다.

4.1.4 요약

Local Storage 기반 3개 프로젝트 모두에서 CSP 그리고 CORS 관련 취약점이 확인되었다.

[Table 8] Local Storage based Project Analysis Results

[표 8] Local Storage 기반 프로젝트 분석 결과

프로젝트	저장 방식	CSP	CORS	JWT 관련 취약점
PERN-Store	Hybrid	0	0	2
MERN E-Commerce	Local Storage	0	0	2
MySQL Registration Passport	Local Storage	0	0	2

4.2 Cookies 기반 JWT 프로젝트의 ZAP 취약점 분석 결과

4.2.1 mern-auth²⁰ 프로젝트

- 탐지 취약점 : CSP Header Not Set
- JWT 관련 취약점 수 : 1개

mern-auth 프로젝트는 Cookie를 활용한 JWT 인증 기능을 지원한다. OWASP ZAP 분석 결과 CSP Header 미설정 취약점이 확인되었다. JWT 관련 취약점은 총 1종으로 분류하였다.

4.2.2 Spree Storefront²¹ 프로젝트

- 탐지 취약점 : CSP Header Not Set, Cross-Domain Misconfiguration, Cookie No HttpOnly Flag, Cookie Without Secure Flag, Cookie without SameSite Attribute
- JWT 관련 취약점 수 : 5개

Spree Storefront 프로젝트는 Cookie를 활용한 JWT 인증 기능을 지원한다. OWASP ZAP 분석 결과 CSP Header 미설정, Cross-Domain Misconfiguration, Cookie의 HttpOnly 미설정, Cookie의 Secure 미설정, Cookie의 SameSite 미적용

¹⁹ <https://github.com/paigen11/mysql-registration-passport>

²⁰ github.com/bradtraversy/mern-auth

²¹ <https://github.com/spree/storefront> (백엔드 기준 스타수 > 50 : <https://github.com/spree/spree>)

오류 취약점이 확인되었다. JWT 관련 취약점은 총 5종으로 분류하였다.

4.2.3 JWT Authentication React²² 프로젝트 분석

- 탐지 취약점 : CSP Header Not Set, Cross-Domain Misconfiguration
- JWT 관련 취약점 수 : 2개

JWT Authentication React 프로젝트는 Cookie를 활용한 JWT 인증 기능을 지원한다. OWASP ZAP 분석 결과 CSP Header 미설정과 CORS 설정 관련 취약점이 확인되었다. 이는 JWT 저장 방식과 연계될 수 있는 잠재적 보안 위험 요소로 분류하였다.

4.2.4 요약

Cookies 기반 3개 프로젝트에서 모두 CSP Header 미설정 취약점을 확인했다. 또한 Spree Storefront 프로젝트와 JWT Authentication React 프로젝트에서는 Cross-Domain Misconfiguration 취약점이 추가 발견되었다. 마지막으로 Spree Storefront 프로젝트에서는 Cookie 보안 속성(HttpOnly, Secure, SameSite) 미설정 취약점이 확인되어 Cookies 프로젝트 중 가장 많은 취약점이 발견되었다.

[Table 9] Cookies based Project Analysis Results

[표 9] Cookies 기반 프로젝트 분석 결과

프로젝트	저장 방식	CSP	CORS	Cookie 보안 속성	JWT 관련 취약점
mern-auth	Cookie	0	X	X	1
Spree Storefront	Cookie	0	0	0	5
JWT Authentication React	Cookie	0	0	X	2

4.3 저장 방식별 비교 분석

4.3.1 취약점 발생 빈도 비교

4.1과 4.2에서 분석한 Local Storage 기반 프로젝트와 Cookies 기반 프로젝트의 탐지 결과를 바탕으로 저장 방식별 보안 특성을 비교하였다. 분석 결과, 저장 방식과 관계없이 CSP Header 미설정 취약점은 모든 프로젝트에서 반복적으로 확인되었다. 이는 JWT 저장 방식의 문제라기보다 웹 애플리케이션들의 보안 정책 설정이 부족하다는 것을 의미한다.

²² https://github.com/wpcodevo/JWT_Authentication_React

[Table 10] Comparison of Vulnerability Frequencies Across Project Groups Based on Token Storage Methods

[표 10] Token 저장 방식에 따라 분류한 프로젝트 그룹 별 취약점 발생 빈도 비교

저장 방식	프로젝트 수	CSP 미설정	CORS 오류	Cookie 보안 속성 미흡	평균 JWT 관련 취약점 수
Local Storage	3	3/3	3/3	해당 없음	2.0
Cookie	3	3/3	2/3	1/3	2.67

추가적으로 언급해보자면, 단순 취약점 개수만 보면 Cookies 기반 프로젝트의 평균 JWT 관련 취약점 수가 더 높게 나타나고 있는데, 이는 Cookies 방식 자체가 더 취약하다는 의미가 아닌 쿠키 속성 설정 여부에 따라 보안 수준의 편차가 크게 발생한다는 의미로 해석하는 것이 적절하다.

4.3.2 Local Storage 방식의 보안 특성

Local Storage 계열로 분류한 프로젝트 모두에서 CSP Header 미설정과 Cross-Domain Misconfiguration 취약점이 확인되었다. Local Storage 방식은 JavaScript를 통해 저장하고 Authorization Header에 직접 포함할 수 있으므로 구현 난이도는 낮은 방식으로 평가된다. 그러나 JavaScript 접근이 가능하다는 특성 때문에 XSS 공격이 발생할 경우 저장된 JWT가 직접 노출될 수 있다.

따라서 Local Storage 방식에서 CSP Header 미설정은 XSS 공격과 결합될 경우 토큰 탈취 가능성을 높이는 요인으로 해석된다. 이는 3.4에서 제시한 구현 난이도 평가 기준 중 XSS 방어 부담이 높은 저장 방식에 해당한다. 즉, Local Storage 방식은 구현 편의성은 높지만, CSP 적용, 입력값 검증, 출력 인코딩과 같은 XSS 설정에 대한 의존도가 큰 방식으로 볼 수 있다.

4.3.3 Cookies 방식의 보안 특성

Cookies 방식은 HttpOnly, Secure, SameSite와 같은 보안 속성을 통해 XSS 및 CSRF 공격을 완화할 수 있다. 특히 HttpOnly 속성이 적용된 쿠키는 JavaScript를 통한 접근이 제한되므로, Local Storage 방식에 비해 XSS 발생 시 토큰 직접 탈취 가능성을 낮출 수 있다.

그러나 Cookies 방식의 보안성은 보안 속성이 올바르게 설정되었을 때 확보된다. 본 연구의 분석 대상인 Spree Storefront 프로젝트에서는 Cookie No HttpOnly Flag, Cookie Without Secure Flag, Cookie without SameSite Attribute 취약점이 함께 확인되었다. 이는 Cookie 방식이더라도 보안 속성이 누락될 경우 XSS 및 CSRF 방어 효과가 약화될 수 있음을 보여준다.

3.4에서 제시한 보안 구현 난이도 평가 기준에 따르면, Cookie 방식은 Set-Cookie 설정, SameSite 속성, Secure 속성, CSRF 방어 설정 등을 함께 고려해야 하므로

Local Storage 방식보다 구현 난이도가 높다. 특히 Cross-Origin 환경에서는 credentials 설정과 Access-Control-Allow-Credentials, Access-Control-Allow-Origin 설정이 함께 요구되므로 설정 오류 가능성이 증가한다. 따라서 Cookie 방식은 저장 방식 자체만으로 안전한 것이 아니라, 보안 속성과 CORS 설정이 올바르게 적용될 때 보안적 이점을 갖는 방식으로 해석된다.

4.3.4. Hybrid(Local Storage + Cookie) 방식의 해석

PERN-Store 프로젝트는 Access Token을 Local Storage에 저장하고 Refresh Token을 Cookie에 저장하는 Hybrid 구조를 사용하였다. 이 방식은 Access Token과 Refresh Token의 역할을 분리한다는 점에서 의미가 있다. Access Token이 탈취되더라도 짧은 만료 시간을 적용하면 피해 범위를 제한할 수 있으며, Refresh Token을 Cookie에 저장함으로써 장기 인증 수단의 직접 노출 가능성을 낮출 수 있다.

그러나 Hybrid 방식은 두 저장 방식의 보안 요구사항을 동시에 고려해야 한다. Access Token이 Local Storage에 저장되는 경우 XSS 공격 발생 시 단기 권한 탈취 가능성이 남아 있으며, Refresh Token이 Cookie에 저장되는 경우 CSRF 방어와 Cookie 보안 속성이 필요하다. 또한 Access Token 만료 처리, Refresh Token 재발급 API, 로그아웃 시 토큰 무효화, Refresh Token Rotation 등의 추가 로직이 필요하다.

따라서 3.4의 구현 난이도 평가 기준에 따르면 Hybrid 방식은 보안성과 피해 제한 측면에서 장점을 가지지만, 인증 흐름과 토큰 갱신 로직이 복잡해 구현 난이도가 높은 방식으로 평가된다.

4.3.5. 보안성 및 구현 난이도 종합 평가

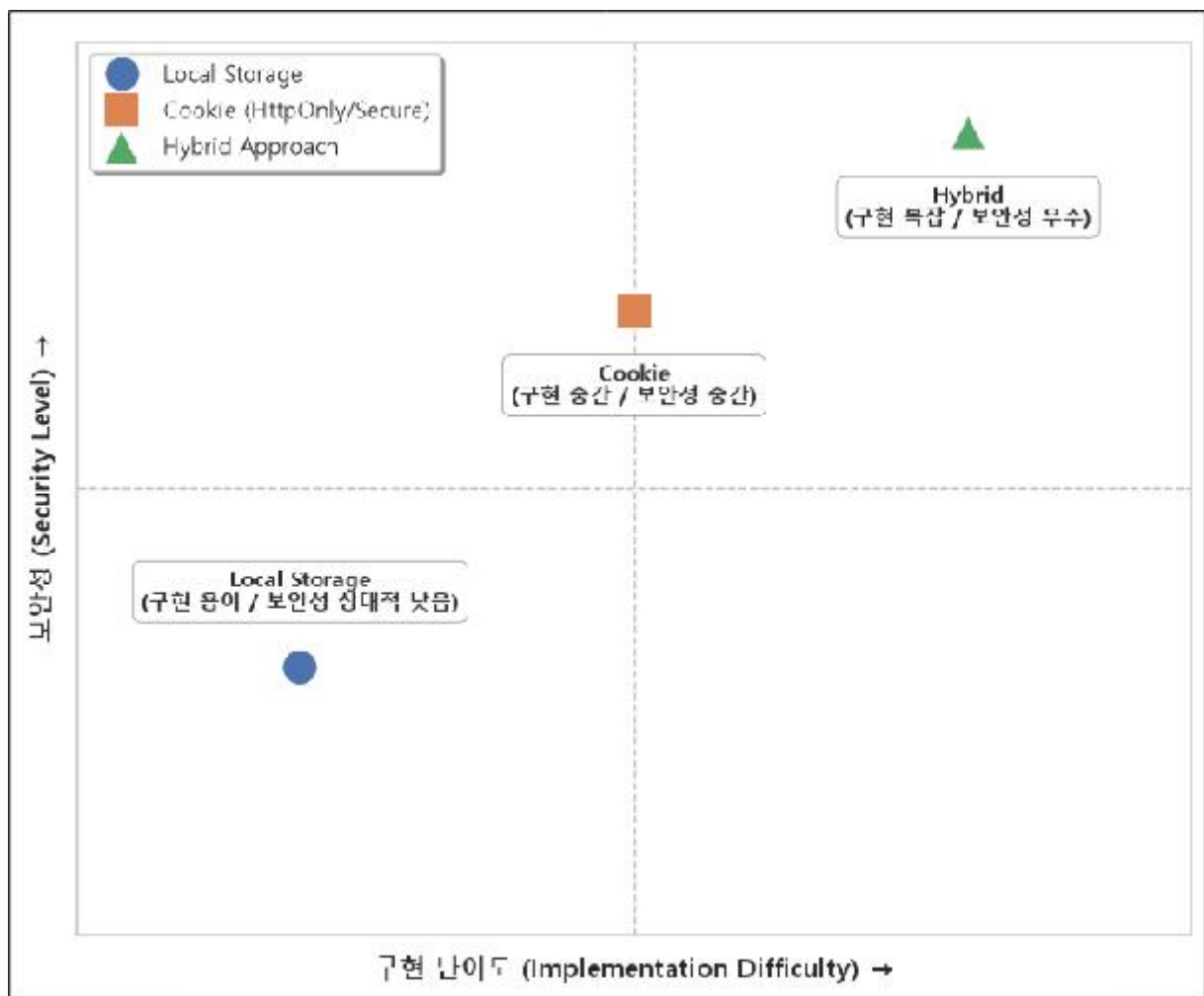
분석 결과 Local Storage 방식은 구현 난이도가 낮고 인증 요청 처리가 단순하다는 장점이 있으나, JavaScript 접근이 가능하므로 XSS 공격 발생 시 토큰 직접 탈취 위험이 크다. Cookies 방식은 HttpOnly, Secure, SameSite 속성을 통해 XSS 및 CSRF 위험을 완화할 수 있으나, 보안 속성 설정이 누락될 경우 장점이 약화된다. 또한 Cross-Origin 환경에서는 CORS와 credentials 설정이 함께 요구되므로 구현 난이도가 증가한다.

Hybrid 방식은 Access Token과 Refresh Token을 분리하여 토큰 탈취 시 피해 범위를 제한할 수 있으나, 토큰 재발급 및 로그아웃 처리 로직이 추가되어 구현 난이도가 가장 높다. 따라서 JWT 저장 방식은 단순한 취약점 개수나 특정 저장소의 우열만으로 판단하기 어렵다. 저장 방식 선택 시에는 XSS 및 CSRF 노출 가능성, Cookie 보안 속성 적용 여부, CORS 설정 복잡도, 토큰 갱신 로직의 구현 부담을 함께 고려해야 한다.

[Table 11] A Comprehensive Comparison of Security and Implementation Complexity Across JWT Storage Methods

[표 11] JWT 저장 방식별 보안성 및 구현 난이도 종합 비교

저장 방식	구현 난이도	XSS 노출 위험	CSRF 노출 위험	토큰 직접 탈취 위험	주요 장점	주요 한계	필수 보안 요소
Local Storage	Low	High	Low	High	구현이 단순하고 Authorization Header 처리 용이	XSS 발생 시 토큰 직접 탈취 가능	CSP, 입력값 검증, 출력 인코딩
Cookies	Medium	Low	Medium~High	Low~Medium	HttpOnly 적용시 JavaScript 접근 차단 가능	Cookie 속성 미설정 시 보안성 급감, Cross-Origin 설정 복잡	HttpOnly, Secure, SameSite, CSRF 방어
Hybrid	High	Medium	Medium	Low	Access/Refresh Token 역할 분리, 피해 범위 제한	구현 복잡도 높음, 토큰 갱신 로직 필요	CSP, Cookie 속성, CORS, Refresh Token Rotation



[Figure 6] Comparison Matrix of Security and Implementation Complexity Across JWT Storage Methods

[그림 6] JWT 저장 방식별 보안성-구현 난이도 비교 매트릭스

4.4 상황별 인증 아키텍처 제안

4.4.1 아키텍처 제안 기준

본 연구에서는 4.3의 저장 방식별 비교 분석 결과를 바탕으로 JWT 인증 아키텍처를 상황별로 제안한다. JWT 저장 방식은 특정 저장소의 절대적 우열로 판단하기보다 서비스 환경, 보안 요구 수준, 구현 복잡도, 토큰 탈취 시 피해 범위까지 함께 고려하여 선택되어야 한다.

본 연구에서는 상황별 인증 아키텍처를 제안하기 위해 다음 기준을 적용한다. 첫째, 서비스가 처리하는 정보의 민감도와 보안 요구 수준을 고려한다. 둘째, Frontend와 Backend가 동일 Origin에서 동작하는지, 서로 다른 Origin으로 분리되어 동작하는지를 고려한다. 셋째, 개발 및 운영 과정에서 감당 가능한 보안 구현 복잡도를 고려한다. 넷째, 토큰 탈취가 발생했을 때 피해 범위를 제한할 수 있는지를 고려한다.

이러한 기준에 따라 본 연구에서는 구현 편의성이 중요한 소규모 서비스, 일반적인 웹 서비스, SPA와 API 서버가 분리된 Cross-Origin 환경, 보안 민감도가 높은 서비스 환경으로 구분하여 각각에 적합한 JWT 인증 아키텍처를 제안한다.

4.4.2 구현 편의성이 중요한 소규모 서비스

구현 편의성이 중요한 소규모 서비스에서는 Local Storage 기반 인증 구조를 제한적으로 고려할 수 있다. Local Storage 방식은 JavaScript를 통해 토큰을 저장하고 Authorization Header에 직접 포함할 수 있으므로 구현이 단순하며, 별도의 Cookie 속성 설정이나 복잡한 인증 흐름을 구성하지 않아도 된다는 장점이 있다. 따라서 학습용 프로젝트, 내부 테스트용 서비스, 프로토타입, 사용자 입력을 저장하거나 화면에 재출력하는 기능이 제한된 단순 조회형 서비스에서는 개발 비용을 줄일 수 있는 방향으로 활용될 수 있다.

다만 Local Storage는 JavaScript 접근이 가능하므로 XSS 공격이 발생할 경우 저장된 JWT가 직접 탈취될 수 있다. 따라서 Refresh Token과 같은 장기 인증 토큰은 저장하지 않는 것이 바람직하며, Access Token의 만료 시간을 짧게 설정하고 CSP, 입력값 검증, 출력 인코딩과 같은 XSS 방어 기법을 함께 적용해야 한다. 개인정보, 결제 정보, 관리자 권한과 같이 민감한 정보를 다루는 서비스에서는 Cookie 기반 또는 Hybrid 인증 구조를 우선적으로 고려해야 한다.

4.4.3 일반적인 웹 서비스 환경

일반적인 웹 서비스 환경에서는 HttpOnly 설정한 Cookie 기반 인증 구조를 우선적으로 고려할 수 있다. 여기서 일반적인 웹 서비스 환경이란 Frontend와 Backend가 동일한 Origin에서 동작하거나, 서버 측에서 화면 렌더링과 API 처리를 함께 수행하여 Cross-Origin 설정 부담이 상대적으로 낮은 구조를 의미한다.

HttpOnly Cookie 방식은 서버가 로그인 성공 시 Set-Cookie 응답 헤더를 통해 JWT를 Cookie에 저장하고, 이후 브라우저가 요청마다 Cookie를 자동으로 전송하는 구조이다. HttpOnly 속성을 적용하면 JavaScript를 통한 Cookie 접근이 제한되므로 Local Storage 방식에 비해 토큰 직접 탈취 위험을 낮출 수 있다. 다만 Cookie가 자동으로 전송되는 특성상 SameSite, CSRF Token, Origin 검증 등의 CSRF 방어 기법을 함께 적용해야 한다.

결과적으로 Same-Origin 기반의 일반 웹 서비스에서는 HttpOnly + SameSite 설정 Cookie 방식이 Local Storage 방식보다 토큰 직접 탈취 위험을 줄일 수 있는 현실적인 대안이 될 수 있다. 다만 해당 구조의 보안성은 Cookie 보안 속성이 누락 없이 올바르게 적용되는 것을 전제로 하므로, Cookie 설정 상태에 대한 지속적인 점검이 필요하다.

4.4.4 SPA와 API 서버가 분리된 Cross-Origin 환경

SPA와 API 서버가 서로 다른 Origin으로 분리된 환경에서는 Cookie 기반 인증 구조를 적용할 수 있으나, Same-Origin 환경보다 더 엄격한 설정 관리가 요구된다. 2.3에서 설명한 것처럼 Cross-Origin 환경에서는 브라우저의 SOP로 인해 다른 Origin과의 통신이 제한되므로, CORS 설정을 통해 인증 요청을 명시적으로 허용해야 한다.

Cross-Origin 환경에서 HttpOnly 옵션 설정 Cookie 기반 인증을 적용할 경우, 서버는 Access-Control-Allow-Origin 헤더를 통해 허용된 Frontend Origin을 명시적으로 지정해야 하며, Access-Control-Allow-Credentials 설정을 통해 인증 정보를 포함한 요청을 허용해야 한다. 클라이언트 측에서도 credentials 또는 withCredentials 옵션을 활성화하여 Cookie가 요청에 포함될 수 있도록 설정해야 한다. 또한 Cookie가 Cross-Origin 요청에서 정상적으로 전송되기 위해서는 SameSite와 Secure 속성 설정도 함께 고려되어야 한다.

이 구조는 SPA 환경에서도 HttpOnly를 설정한 Cookie를 활용하여 JavaScript를 통한 토큰 직접 접근을 제한할 수 있다는 장점이 있다. 그러나 CORS, credentials, SameSite, Secure 설정 중 하나라도 누락되면 Cookie가 저장되지 않거나 요청에 포함되지 않아 인증 흐름이 정상적으로 동작하지 않을 수 있다. 따라서 Cross-Origin Cookie 기반 인증 구조는 보안 설정 점검을 전제로 적용하는 것이 바람직하다.

4.4.5 보안 민감도가 높은 서비스 환경

개인정보, 결제 정보, 관리자 권한과 같이 보안 민감도가 높은 기능을 제공하는 서비스에서는 Access Token과 Refresh Token을 분리한 Hybrid 인증 구조를 우선적으로 고려할 수 있다. Hybrid 구조는 Access Token을 API 접근 권한 확인에 사용하고, Refresh Token을 Access Token 재발급에 사용하는 방식으로 토큰의 역할을 분리한다.

이 구조에서는 Access Token의 만료 시간을 짧게 설정하여 탈취 시 사용 가능한 시간을 제한하고, Refresh Token은 JavaScript 접근이 제한되는 HttpOnly Cookie에 저장하는 것

이 바람직하다. 또한 Secure, SameSite와 같은 Cookie 보안 속성, CSRF 방어, CORS 설정을 함께 고려해야 한다.

보안 요구 수준이 높은 서비스에서는 Refresh Token Rotation을 적용하여 재발급 시마다 새로운 Refresh Token을 발급하고, 이전 Refresh Token이 재사용되는 경우 탈취 가능성을 의심하여 세션을 종료하는 방식도 고려할 수 있다.

Hybrid 인증 구조는 Local Storage 방식이나 단순 Cookie 방식보다 구현 난이도가 높지만, 토큰의 역할 분리와 짧은 Access Token 만료 시간을 통해 토큰 탈취 시 피해 범위를 줄일 수 있다는 장점이 있다.

4.4.6 상황별 인증 아키텍처 종합 비교

앞서 제안한 상황별 인증 아키텍처를 종합하면[표 12]과 같다. 본 연구에서 제안하는 인증 구조는 특정 저장 방식 하나를 모든 환경에 적용하는 것이 아니라, 서비스의 보안 요구 수준, Origin 구성 방식, 구현 복잡도, 토큰 탈취 시 피해 제한 가능성을 함께 고려하여 선택하는 것을 전제로 한다.

[Table 12] Comparison of Context-Specific JWT Authentication Architectures

[표 12] 상황별 JWT 인증 아키텍처 종합 비교

서비스 환경	권장 인증 구조	Access Token 저장 및 처리 방식	Refresh Token 저장 방식	주요 보안 설정	선택 이유 및 고려사항
구현 편의성이 중요한 소규모 서비스	Local Storage 기반 인증	Local Storage 저장 후 Authorization Header에 포함	저장하지 않거나 사용을 제한	CSP, 입력값 검증, 출력값 인코딩, 짧은 만료 시간	구현이 단순하여 학습용 프로젝트, 내부 테스트, 프로토타입에 적합하다. 단, XSS 발생 시 토큰 직접 탈취 위험이 있으므로 보안 민감 서비스에는 적합하지 않다.
일반적인 웹 서비스	Same-Origin HttpOnly Cookie 기반 인증	HttpOnly Cookie를 통해 자동 전송	필요 시 HttpOnly Cookie 저장	HttpOnly, Secure, SameSite, CSRF Token, Origin 검증	Same-Origin 환경에서는 CORS 설정 부담이 낮고, HttpOnly 속성을 통해 JavaScript 기반 토큰 접근을 제한할 수 있다. 단, Cookie 자동 전송 특성상 CSRF 방어가 필요하다.
SPA와 API서버가 분리된 Cross-Origin 환경	Cross-Origin Cookie 기반 인증	HttpOnly Cookie를 통해 자동 전송	필요 시 HttpOnly Cookie 저장	CORS, Access-Control-Allow-Origin, Access-Control-Allow-Credentials, credentials 설정, Secure, SameSite, CSRF 방어	Frontend와 Backend가 분리된 구조에서도 Cookie 기반 인증을 적용할 수 있다. 다만 CORS와 credentials 관련 설정이 복잡하므로 설정 오류 여부를 지속적으로 점검해야 한다.
보안 민감도가 높은 서비스	Hybrid 인증 구조	짧은 수명의 Access Token 사용, 메모리 저장 우선 고려	HttpOnly Cookie 저장	HttpOnly, Secure, SameSite, CSRF 방어, CORS, Refresh Token Rotation, 로그아웃 시 토큰 무효화	Access Token과 Refresh Token의 역할을 분리하여 토큰 탈취 시 피해 범위를 줄일 수 있다. 구현 난이도는 높지만 개인정보, 결제 정보, 관리자 권한을 다루는 서비스에 적합하다.

종합적으로 Local Storage 방식은 구현 편의성이 중요한 제한적 환경에서 활용될 수 있으나, XSS 공격에 따른 토큰 직접 탈취 위험을 고려해야 한다. Same-Origin 기반의 일반 웹 서비스에서는 HttpOnly Cookie 방식이 현실적인 대안이 될 수 있으며, Cross-Origin 환경에서는 CORS와 credentials 설정을 포함한 추가 보안 구성이 요구된다. 보안 민감도가 높은 서비스에서는 Access Token과 Refresh Token을 분리한 Hybrid 구조를 적용하여 토큰 탈취 시 피해 범위를 제한하는 것이 바람직하다. 따라서 JWT 인증 아키텍처는 저장 방식 자체의 우열보다 서비스 환경과 보안 요구 수준에 따라 선택되어야 한다.

5. 결론

본 연구는 브라우저 환경에서 JWT 인증 정보를 저장하는 방식에 따라 발생할 수 있는 보안 취약점을 분석하고, 서비스 환경에 따라 적용 가능한 인증 아키텍처를 제안하는 것을 목적으로 하였다. 이를 위해 JWT 인증 방식을 사용하는 GitHub 오픈소스 프로젝트를 Local Storage 기반 프로젝트와 Cookie 기반 프로젝트로 구분하고, OWASP ZAP을 활용하여 CSP Header 미설정, Cross-Domain Misconfiguration, Cookie 보안 속성 미흡 등 JWT 인증 구조와 관련될 수 있는 취약점을 분석하였다.

분석 결과, 저장 방식과 관계없이 CSP Header 미설정 취약점이 반복적으로 확인되었으며, 이는 JWT 저장 방식 자체의 문제라기보다 웹 애플리케이션 전반의 보안 정책 설정 미흡과 관련된 문제로 해석할 수 있다. Local Storage 방식은 구현이 단순하고 Authorization Header를 통한 인증 요청 처리가 용이하다는 장점이 있으나, JavaScript 접근이 가능하므로 XSS 공격 발생 시 토큰이 직접 노출될 수 있다는 한계를 가진다. Cookies 방식은 HttpOnly, Secure, SameSite와 같은 보안 속성을 통해 토큰 직접 탈취 위험을 낮출 수 있으나, 해당 속성이 올바르게 설정되지 않을 경우 보안적 장점이 약화될 수 있다. 또한 Cross-Origin 환경에서는 CORS와 credentials 설정이 함께 요구되므로 설정 오류 가능성과 구현 복잡도가 증가한다.

이러한 분석을 바탕으로 본 연구에서는 특정 JWT 저장 방식이 모든 환경에서 우수하다고 판단하기보다, 서비스 환경과 보안 요구 수준에 따라 인증 구조를 선택해야 한다고 보았다. 구현 편의성이 중요한 소규모 서비스에서는 Local Storage 기반 인증 구조를 제한적으로 고려할 수 있으나, 장기 인증 토큰 저장은 지양하고 XSS 방어 설정을 함께 적용해야 한다. 일반적인 Same-Origin 기반 웹 서비스에서는 HttpOnly를 설정한 Cookie 방식이 현실적인 대안이 될 수 있으며, SPA와 API 서버가 분리된 Cross-Origin 환경에서는 CORS, credentials, SameSite, Secure 설정을 함께 관리해야 한다. 개인정보, 결제 정보, 관리자 권한과 같이 보안 민감도가 높은 서비스를 대상으로는 Access Token과 Refresh Token을 분리한 Hybrid 구조를 우선적으로 고려할 수 있다.

다만 본 연구는 제한된 수의 오픈소스 프로젝트를 대상으로 OWASP ZAP 기반 자동화 분석을 수행하였으므로, 모든 JWT 인증 환경을 일반화하는 데에는 한계가 있다. 또한 탐지된 취약점이 실제 토큰 탈취나 권한 도용으로 이어지는 과정을 직접 공격 실험으로 검증하지는 못하였다. 향후 연구에서는 더 많은 프로젝트를 대상으로 분석 범위를 확대하고, XSS 및 CSRF 공격 시나리오를 직접 구성하여 저장 방식별 실제 공격 가능성과 피해 범위를 정량적으로 비교할 필요가 있다. 이를 통해 JWT 저장 방식 선택에 대한 보다 구체적인 보안 기준과 실무 적용 가능한 인증 아키텍처 가이드라인을 제시할 수 있을 것이다.

References

- [1] (<https://datatracker.ietf.org/doc/html/rfc7519>, June 9 (2026)
- [2] ChaeEun Lee and Sanghoon Jeon. (2024). A Study on the Security Performance of JWT Token Signature Algorithms. Journal of convergence security, 24(4), 3-10.
- [3] (<https://datatracker.ietf.org/doc/html/rfc8725>, June 9 (2026)
- [4] (https://en.wikipedia.org/wiki/HTTP_cookie, June 11(2026)
- [5] (<https://developer.mozilla.org/ko/docs/Glossary/Origin>, June 12(2026)
- [6] (https://developer.mozilla.org/ko/docs/Web/Security/Defenses/Same-origin_policy, June 12(2026)
- [7] (<https://datatracker.ietf.org/doc/html/rfc6749#section-1.5> June 13(2026)
- [8] Kim, J., Kwak, S., & Junho, B. (2022). New Hybrid S-JWT Server Design and Implementation. Journal of Next-generation Convergence Technology Association, 6(8), 1365-1371.
<https://doi.org/10.33097/JNCTA.2022.06.08.1365>